Table des matières

Labo 04 — Charger, décharger, et suivre les changements 1
Prérequis 1
Étape 0 — Poser le contexte 1
Étape 1 — Un format de fichier et un stage 2
Étape 2 — Décharger vers le stage (COPY INTO <emplacement>) 2
Étape 3 — Recharger dans une table (COPY INTO <table>) 2
Étape 4 — L'idempotence (et comment la forcer) 3
Étape 5 — La gestion d'erreurs (VALIDATION_MODE, ON_ERROR) 3
Étape 6 — Suivre les changements avec un Stream (CDC) 4
Étape 7 *(aperçu)* — Une Dynamic Table 4
Nettoyage 5
Récapitulatif 5

Labo 04 — Charger, décharger, et suivre les changements

Module 4/8 — Formation Snowflake (préparation SnowPro Core, COF-C03)

But du labo — *Voir tourner* le domaine 3, sans fichier externe : on décharge un échantillon vers un stage interne, on le recharge dans une table (`COPY INTO`), on éprouve la gestion d'erreurs et l'idempotence, puis on capture des changements avec un Stream et on goûte une Dynamic Table. Comptez 30 à 40 minutes. Tout se fait dans un worksheet Snowsight.

Prérequis

- Le Cours 00 est fait : entrepôt `SNOWPRO_WH` + base `SNOWPRO_DB.SANDBOX`.
- Astuce : la boucle « décharger puis recharger » nous évite d'avoir un fichier sur le disque, on fabrique nos fichiers depuis Snowflake, dans un stage interne.

Étape 0 — Poser le contexte

© 2026 iAlexMG. Tous droits réservés.

```sql
USE ROLE SYSADMIN;
USE WAREHOUSE SNOWPRO_WH;
USE DATABASE SNOWPRO_DB;
USE SCHEMA SANDBOX;
```

Étape 1 — Un format de fichier et un stage

Théorie : `Theorie_04_SNOW.md`, sections 2 et 3.

```sql
-- Comment LIRE nos fichiers CSV (délimiteur, en-tête, guillemets).
CREATE OR REPLACE FILE FORMAT ff_csv
  TYPE = CSV
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"';

-- Un stage interne NOMMÉ : l'aire d'atterrissage de nos fichiers.
CREATE OR REPLACE STAGE mon_stage FILE_FORMAT = ff_csv;
```

Ce que vous observez. Deux objets créés. Le stage est vide pour l'instant, on va le remplir en déchargeant.

Étape 2 — Décharger vers le stage (`COPY INTO <emplacement>`)

Théorie : `Theorie_04_SNOW.md`, section 5.

```sql
-- On exporte 1000 clients d'exemple en CSV, dans le stage.
COPY INTO @mon_stage/clients/
  FROM (SELECT C_CUSTKEY, C_NAME, C_MKTSEGMENT
        FROM   SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER
        LIMIT  1000)
  FILE_FORMAT = (FORMAT_NAME = ff_csv)
  HEADER = TRUE
  OVERWRITE = TRUE;

-- Que contient le stage, maintenant ?
LIST @mon_stage;
```

Ce que vous observez. `LIST` montre un fichier (p. ex. `clients/data_0_0_0.csv.gz`), car Snowflake compresse d'office. Vous venez de faire un déchargement (*unload*) : une requête → des fichiers.

Étape 3 — Recharger dans une table (`COPY INTO <table>`)

© 2026 iAlexMG. Tous droits réservés.

Théorie : `Theorie_04_SNOW.md`, section 4.

```
-- La coquille vide (Cours 01) ...
CREATE OR REPLACE TABLE clients_charges (custkey NUMBER, nom STRING, segment STRING);

-- ... que COPY remplit depuis le stage (l'en-tête est sauté grâce à SKIP_HEADER=1).
COPY INTO clients_charges
  FROM @mon_stage/clients/
  FILE_FORMAT = (FORMAT_NAME = ff_csv);

SELECT COUNT(*) FROM clients_charges;    -- attendu : 1000
SELECT * FROM clients_charges LIMIT 5;
```

Ce que vous observez. 1000 lignes. Le mapping se fait par position (colonne 1 → `custkey`, 2 → `nom`, 3 → `segment`). `CREATE TABLE` a fait la coquille ; `COPY` l'a remplie.

Étape 4 — L'idempotence (et comment la forcer)

Théorie : `Theorie_04_SNOW.md`, section 4.

```
-- Rejouez EXACTEMENT le même COPY : le fichier est déjà chargé -> il est IGNORÉ.
COPY INTO clients_charges FROM @mon_stage/clients/ FILE_FORMAT = (FORMAT_NAME = ff_csv);
SELECT COUNT(*) FROM clients_charges;    -- toujours 1000 (aucun doublon)

-- FORCE = TRUE ordonne de tout RECHARGER (là, ça double) :
COPY INTO clients_charges FROM @mon_stage/clients/ FILE_FORMAT = (FORMAT_NAME = ff_csv) FORCE = TRUE;
SELECT COUNT(*) FROM clients_charges;    -- attendu : 2000

-- On remet propre pour la suite.
TRUNCATE TABLE clients_charges;
```

Ce que vous observez. Le 2e `COPY` charge 0 fichier : Snowflake retient les fichiers déjà chargés (métadonnées de charge, ~64 jours), donc rejouer un `COPY` ne double pas les données, sauf `FORCE = TRUE`. C'est ce qui rend un pipeline rejouable sans danger.

Étape 5 — La gestion d'erreurs (`VALIDATION_MODE`, `ON_ERROR`)

Théorie : `Theorie_04_SNOW.md`, section 4.

```
-- On fabrique un cas d'erreur : on décharge du TEXTE (des noms)...
COPY INTO @mon_stage/noms/
  FROM (SELECT C_NAME FROM SNOWFLAKE_SAMPLE_DATA.TPCH_SF1.CUSTOMER LIMIT 100)
  FILE_FORMAT = (TYPE = CSV) HEADER = FALSE OVERWRITE = TRUE;
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- ... qu'on tentera de charger dans une colonne NUMÉRIQUE (le texte ne rentrera pas).
CREATE OR REPLACE TABLE nombres (n NUMBER);

-- Test À BLANC : montre les erreurs SANS rien écrire.
COPY INTO nombres FROM @mon_stage/noms/
  FILE_FORMAT = (TYPE = CSV)
  VALIDATION_MODE = 'RETURN_ERRORS';

-- Chargement TOLÉRANT : saute les lignes fautives, garde ce qui passe.
COPY INTO nombres FROM @mon_stage/noms/
  FILE_FORMAT = (TYPE = CSV)
  ON_ERROR = 'CONTINUE';
SELECT COUNT(*) FROM nombres;            -- attendu : ~0 (tous les noms sont rejetés)
```

Ce que vous observez. `VALIDATION_MODE = 'RETURN_ERRORS'` liste les erreurs sans charger, c'est le filet avant d'ingérer pour de vrai. `ON_ERROR = 'CONTINUE'` saute les lignes fautives (ici, toutes) au lieu d'annuler le chargement entier (le défaut, `ABORT_STATEMENT`).

Étape 6 — Suivre les changements avec un Stream (CDC)

Théorie : `Theorie_04_SNOW.md`, section 6.

```sql
CREATE OR REPLACE TABLE commandes (id INT, montant NUMBER);
CREATE OR REPLACE STREAM flux_commandes ON TABLE commandes;   -- suit les changements de la table

SELECT COUNT(*) FROM flux_commandes;     -- attendu : 0 (rien n'a changé depuis la création)

-- On modifie la table...
INSERT INTO commandes VALUES (1, 100), (2, 250);
UPDATE commandes SET montant = 300 WHERE id = 2;

-- ...le stream a CAPTURÉ le résultat net des changements.
SELECT id, montant, METADATA$ACTION, METADATA$ISUPDATE FROM flux_commandes;
```

Ce que vous observez. Le stream expose les lignes changées avec `METADATA$ACTION` (`INSERT` / `DELETE`) et `METADATA$ISUPDATE`. Lire un stream ne le consomme pas ; c'est une DML qui lit le stream qui avance son curseur. C'est la brique du CDC, souvent couplée à une Task (« toutes les 5 min, traite ce qui a changé »).

Étape 7 *(aperçu)* — Une Dynamic Table

Théorie : `Theorie_04_SNOW.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

```sql
-- Déclaratif : vous écrivez la requête cible, Snowflake maintient la table à jour seul.
CREATE OR REPLACE DYNAMIC TABLE resume_commandes
  TARGET_LAG = '1 minute'
  WAREHOUSE  = SNOWPRO_WH
  AS SELECT id, montant * 2 AS montant_double FROM commandes;

ALTER DYNAMIC TABLE resume_commandes REFRESH;   -- force un rafraîchissement immédiat
SELECT * FROM resume_commandes;
```

Ce que vous observez. La Dynamic Table se recalcule seule selon son `TARGET_LAG` (ici 1 min) ; `REFRESH` force la mise à jour tout de suite. Pas de Stream ni de Task à écrire : la transformation est déclarative.

Nettoyage

```sql
DROP DYNAMIC TABLE IF EXISTS resume_commandes;
DROP STREAM        IF EXISTS flux_commandes;
DROP TABLE         IF EXISTS commandes;
DROP TABLE         IF EXISTS nombres;
DROP TABLE         IF EXISTS clients_charges;
DROP STAGE         IF EXISTS mon_stage;      -- vide aussi les fichiers déchargés
DROP FILE FORMAT   IF EXISTS ff_csv;
```

Récapitulatif

- ☐ J'ai créé un format de fichier et un stage interne.
- ☐ J'ai déchargé (`COPY INTO @stage`) puis rechargé (`COPY INTO <table>`) des données.
- ☐ J'ai constaté l'idempotence du `COPY` (et l'effet de `FORCE`).
- ☐ J'ai vu `VALIDATION_MODE` et `ON_ERROR` à l'œuvre sur des lignes fautives.
- ☐ J'ai capturé des changements avec un Stream (`METADATA$ACTION`).
- ☐ J'ai créé une Dynamic Table qui se maintient à jour toute seule.

Tout coché, la donnée entre et sort, en lot comme en continu. Cap sur le Cours 05, performance, requêtes et transformation, le 2e domaine le plus lourd de l'examen.

© 2026 iAlexMG. Tous droits réservés.